

Drawing demo

Marko Guberina

November 26, 2025

1 Introduction

The demo is for the robot to replicate a drawing you make on the computer. The complete demo works as follows:

1. You are prompted with a blank canvas on which you are to draw an image. You are presumably close to the robot.
2. The robot then comes to you to pick up a marker with which to draw.
3. The robot calibrates the length of the marker (the grasp and marker size are unknown) by touching the board once.
4. The robot then proceeds to create the drawing. It treats the drawing trajectory as a dynamic motion primitive (DMP). It uses impedance to maintain contact with the whiteboard.
5. The robot comes back to pick up an eraser.
6. It calibrates the length of the eraser.
7. It then "draws" a wiping picture. But the stiffness is higher, and the target is now more into the board.

The main file is `drawing_from_input_drawing.py`.

2 How it works on 2025-11-26

2.1 Things to be done before you can run the demo

2.1.1 Find a whiteboard and a few markers

Get a whiteboard and some whiteboard markers. Preferably an old whiteboard out of use, as it is very possible (likely even) the robot will make dents in it. Also, the robot will certainly ruin 1 marker, and there's a high possibility it will be more than one. This is because, at least on UR5e, the force-torque sensor is not very sensitive, so we have to shove the marker into the board with forces

between 2.5-5N (the force isn't controlled directly, but more on this later). And the drawing is done with the marker being perpendicular.

The reasons behind this is that we don't assume great board calibration nor that the board is perfectly flat. At least ours isn't.

2.1.2 Manually define a sensible work pose for the robot

Basically find a home position from which Cartesian controllers can easily access the board. This is very important because the demo does not account for any kind of collision constraints, it just assumes that everything will be fine. From experience there were no problems with this as the robot moves relatively little to do the drawing.

Write this joint configuration down.

2.1.3 Marker pickup/handover

To add some flare to the demo, there is a feature for the robot to come to you so that you give it the marker, or cloth to wipe the board with. The idea being that you sit behind a computer close to the robot. It would be very easy to do a pick and place instead (feel free to put this in), this is just what's in the code right now.

The handover is executed as joint-space motion, so you need to provide joint angles constituting the path. The current expected way to do this is to run the `convenience_toolbox/freedrive_v2.0.py`. It puts the robot to freedrive/leadthrough and allows you to save angles by pressing a button on the keyboard. The initial pose for the robot should be the "drawing home" you defined in the previous step. These angles are then fed into a DMP which executes the trajectory. The gripper then opens, waits a few seconds, and then closes. The same exact trajectory, only reversed, is executed to go back. NOTE that you need to manually rename the file outputted from freedrive to `./data/from_writing_to_handover.pickle_save`. Yes, this should be in parameters, and yes this step should be automated (or at least called from the main file).

Note This option is off by default. Enable it with `-pick-up-marker`.

2.1.4 Whiteboard (plane) pose calibration

Before any drawing, board calibration needs to be performed. Do this with `-calibrate-plane` on the main file. The program will guide to the setup, but the point is you go into freedrive and put the end-effector to the top-left corner of the board. The program then samples from the board point to the right and down by touch the board. It updates the orientation estimate of the end-effector as it goes, so that you (eventually) get a flat contact point to the board. It then applies a linearly weighted least-squares fit to the accumulated points, biasing the more recent samples.

Of course, you need to run calibration only once. The pose is saved into `./parameters/plane_pose.pickle`. At the moment the actual file being used is `./parameters/plane_pose.pickle_save`, meaning you have to manually rename it. This is a stupid atavism and will be removed asap (no promises at the time of writing though).

2.1.5 Do a few runs and tune the parameters

The defaults are sane, but the particulars always depend on the concrete robot and other equipment used. What to pay attention to is described in the following section as it was more natural to write that way.

2.2 Normal demo operation

First you are prompted to draw the image. It has to be a single continuous line. You can redo the drawing by just attempting to draw a new one. Accept by pressing Enter. You can also reuse the previous drawing if you selected `-no-draw-new` (this is handy for board wiping, more on that later).

The next action is the optional handover/pickup. It then possibly first does a `moveJ` to home, not sure. **The robot then goes to a point above the board** (as in away from the board) which was the first calibration point. **It then go straight down into the board to touch it once**, and it immediately goes back up to the same point. This is done to calibrate the length of the marker which is unknown both due to the handover, and, even if you didn't go for the handover, due to the possible in-hand slippage during writing (it can and does happen, but not super often - the issue is that this number need to be precise).

With this, we can map the drawing onto the board, i.e. create the SE3 reference out of it. Once this is done, a simulation is run to `moveL` to every point of the reference and record the joint angles. This takes about 1-10 seconds depending on path length. **IMPORTANT TO UNDERSTAND POSSIBLE FAILURES** From these joint angles a DMP is formed. The timing for the DMP is provided with `-tau0`. There is automatic scaling if the robot can't make it in time. But this distorts the image quite a bit. By this I mean it does not work at all, but at least the robot does not explode (as in do something random and violent). Anyhow, a 10 second default is chosen. This is enough for basic shapes which take little time to draw, which is what is assumed to be default (I personally always draw hearts). Other possible distortions are due to board bendiness and bad f/t readings.

We then do a compliantMoveL to the beginning of the drawing. Drawing starts immediately after. Impedance in the form of direct force feedback is added to the DMP. Only the end-effector z axis is compliant, the rest is fully stiff. The reference path is inside the board. How much is determined with `-mm-into-board`, with 3mm being the default. In our experiments this results in 2.5-5N of force. Look at the wrench plot to see how much you use (NOTE that the wrench_{used} i.e. filtered looks bigger as we multiply the

rotational dofs as 1-10 linear-rotational ratio is generally nice compliance). Remember to use `-save-log`, `-index-runs` and `-run-name` to save the log as they are not saved by default. This way you can analyse this in peace, instead of while the robot is running. **IMPORTANT TO UNDERSTAND POSSIBLE FAILURES** If the robot does something weird like randomly push to hard into the board, and then go back, or just go back, this is almost certainly a faulty f/t sensor reading. While you can make this better with better filtering, be wary of garbage in, garbage out - this is the only sensing we use in the demo, and if this is off there's fundamentally no way to save it.

Finally, the robot does compliantMoveL to go above the board. The program then exits.

However, this is not necessarily the end of the demo! **You can re-run the script, but with `-pick-up-marker -board-wiping -mm-into-board=XY -no-draw-new` to pick up a cloth to wipe the board with, and wipe the drawing** by tracing the exact drawing path back! Can't be more efficient than that. How much `-mm-into-board` you need depends on the cloth used. We got the best results with a strongly folded set of kitchen towels as this was just compliant enough (so it's rather sturdy, but not a rigid body). You might also experiment with `-alpha` which determines the amount of compliance (higher means more compliance), but in our experience just changing the reference was enough.

3 Limitations

- The drawing has to be a single continuous line.
 - A different program for drawing is necessary (or the current one needs to be extended).
 - The same controller could be applied for each line, but logic to go line by line needs to be implemented.
 - To avoid it being excruciatingly slow, the default timing for a line needs to be replaced with a heuristic.
- The change of the reference due to impedance distorts the picture. Intricate features won't happen. A different controller would need to be written to more aggressively favour path following over timing (or disregard timing entirely).
- The performance depends on hardware and on calibration a bit more than it should. Some of the calibration is automated, but one still needs to devote around 30min to tune all the parameters for the best performance.

Things to improve/re-solve the problem:

1. Allow the viz option to draw every path point.

2. Add a non-convergence detection to compliantMoveL, so that if you didn't move much, and you're already close to the goal, just exit - you will never converge due to the f/t noise being stronger than the signal to converge. So `if err < 0.01 and avg(v) < 0.01: return`
3. Switch DMP to Cartesian space. This way you don't need to recompute everything to joint space - faster! BONUS: you can skip the offset, and just get it from the first point to be drawn — then update the rest of the reference.
4. Implement better kinesthetic teaching so that the handover is easier to do.
5. Try doing it with basic Cartesian path following instead (skip the DMP).
6. Optimize moveLs so that they're faster (the default point-to-point velocity profile is kinda ass).

Micro quality of life improvements (maybe):

- Replace tau0 (DMP traj) from an argument into a heuristic depending on the length of the trajectory. Anything sensible will be better than the current version which already is a heuristic and only works for short drawings.
- Add an automatic restart if you observe a f/t sensor crap out.
- Handover joint angles definition via kinesthetic teaching should be added into the main program with an `-initialize-demo` argument which is `False` by default. It would also automatically set the calibration flag to `True`.
- In retrospect, it would be better if the calibration process starts with the end-effector touching the board, and then go up a bit, instead of starting from the top. This way the initial estimate is better. Also, maybe it would be better to simply discard the initial points, idk. It does work so this is kind of pointless optimization.
- The output of the